

1.3 Package Management

Edge AI Engineering Bootcamp | Analog Data

Why This Matters

Software on Linux doesn't just appear — it has to be installed, updated, and sometimes locked in place so it doesn't accidentally change. This module covers how the Raspberry Pi installs system software (`apt`), how it installs Python software (`pip3`), and how to configure the Pi's hardware settings and check if it's overheating. Think of this as learning the "settings and installer" side of your Pi.

1. `apt` Deep Dive — The System's App Store

`apt` (Advanced Package Tool) is how Raspberry Pi OS installs, updates, and removes software — things like Python itself, git, or system libraries.

Simple way to think about it: if the Raspberry Pi OS were a phone, `apt` is its app store. Except instead of tapping icons, you type commands.

```
sudo apt update           # refresh the list of available packages (doesn't install
anything yet)
sudo apt upgrade -y       # actually install the newer versions of everything
sudo apt install git      # install one specific package (git)
sudo apt remove git       # remove a package
sudo apt autoremove       # clean up leftover packages nothing else needs
anymore
```

Important distinction to remember:

- `apt update` = "go check what's new" (like refreshing an app store page)
- `apt upgrade` = "actually download and install the updates"

You always run `update` before `upgrade` — if you skip it, you might be upgrading based on outdated information.

`apt-cache show` — Look Before You Install

Before installing something, you can inspect it — like reading the description of an app before downloading it.

```
apt-cache show python3
```

This shows you:

- What the package actually does

- Its version number
- Its size
- What other packages it depends on

Example output (shortened):

```
Package: python3
Version: 3.11.2-1+b1
Depends: python3.11 (>= 3.11.2-6~)
Description: interactive high-level object-oriented language
```

Why this matters: On a small device like a Raspberry Pi, storage and RAM are limited. Checking what a package needs *before* installing it helps avoid surprises — like installing something that pulls in 500MB of dependencies you didn't expect.

apt-mark hold — Freeze a Package So It Never Updates

Sometimes you don't want a package to update automatically — maybe your project depends on one exact version, and a newer version might break something.

```
sudo apt-mark hold python3      # lock python3 at its current version
sudo apt-mark unhold python3    # remove the lock, allow updates again
apt-mark showhold              # see everything currently locked
```

Real-world example: Imagine your camera + AI script only works correctly with `libcamera` version 0.1.0. If you run `apt upgrade` and it silently updates `libcamera` to 0.2.0, your whole project might break. `apt-mark hold libcamera` prevents that specific package from ever being touched by future upgrades, while everything else updates normally.

2. pip3 — Installing Python Packages

While `apt` installs system-wide software, `pip3` installs Python-specific packages — libraries like `numpy`, `opencv-python`, or `tensorflow`.

```
pip3 install numpy      # install a Python package
pip3 uninstall numpy    # remove it
pip3 list               # see everything currently installed
pip3 show numpy         # see details about one installed package
```

--user vs System-Wide Installs

```
pip3 install --user numpy
```

What `--user` actually does: instead of installing the package system-wide (where every user and every program on the Pi can see it), it installs it **only into your own user account's personal folder**.

System-wide install:	--user install:
/usr/lib/python3/	/home/pi/.local/lib/python3/
└─ numpy/	└─ numpy/
(everyone can use it)	(only 'pi' user can use it)

Why this matters: installing system-wide often requires `sudo`, and can conflict with packages the operating system itself depends on. Using `--user` keeps your personal Python packages separate and safer, and doesn't need `sudo` at all.

--break-system-packages (Bookworm-Specific Change)

Starting with **Raspberry Pi OS "Bookworm"** (based on Debian 12), Python enforces a new safety rule called **PEP 668**. By default, `pip3` now **refuses** to install packages directly into the system Python environment.

If you try a plain install and get blocked, you'll see something like:

```
error: externally-managed-environment
```

This is Python protecting itself. The OS itself relies on certain Python packages to function — if you casually `pip install` something that conflicts with a package the OS needs, you could break the entire system.

Three ways to handle this, from safest to most risky:

Approach	Command	When to Use
✅ Best practice	Use a virtual environment (<code>python3 -m venv myenv</code>)	Almost always — keeps your project's packages completely isolated
⚠️ Acceptable	<code>pip3 install --user package_name</code>	Quick personal scripts, non-critical tools
🚫 Risky	<code>pip3 install --break-system-packages package_name</code>	Only when you fully understand the risk — forces the install anyway, bypassing the safety check

```
# Recommended: create an isolated environment first
python3 -m venv myproject-env
source myproject-env/bin/activate
pip3 install numpy opencv-python

# Only if you truly need to override the safety check:
pip3 install --break-system-packages some-package
```

Simple rule to give students: *If `pip3` refuses to install something on Bookworm, that's not a bug — it's the OS protecting itself. Use a virtual environment instead of forcing it.*

3. /boot/config.txt — The Pi's Hardware Control Panel

This file controls **low-level hardware settings** — things that are decided before Linux even fully boots. On newer OS versions, the actual path is `/boot/firmware/config.txt` , but it works the same way.

```
sudo nano /boot/config.txt      # edit the file
```

Think of `config.txt` like the **BIOS settings** on a regular PC, except it's just a plain text file you can edit yourself.

GPU Memory Split

The Pi shares one pool of RAM between the CPU and GPU. This setting decides how much goes to the GPU.

```
gpu_mem=128
```

Use Case	Recommended <code>gpu_mem</code>
Headless (no display, pure AI/server work)	16 — give almost everything to the CPU
Camera + light graphics	64 – 128
Desktop GUI or heavy video work	128 – 256

Why this matters for Edge AI: If you're running a headless AI inference project with no monitor attached, setting `gpu_mem=16` frees up more RAM for your CPU-based model to actually use.

Enabling I2C / SPI / UART / Camera

Sensors and cameras communicate over specific hardware interfaces that are **disabled by default** for security and power-saving reasons.

```
dtparam=i2c_arm=on      # enable I2C (used by many sensors: temperature, IMU,
OLED displays)
dtparam=spi=on           # enable SPI (used by displays, some sensors, ADCs)
enable_uart=1            # enable UART (serial communication, e.g., GPS
modules)
camera_auto_detect=1     # enable the official camera module
```

Real example: If you connect a temperature sensor over I2C and it's not detected, the very first thing to check is whether `dtparam=i2c_arm=on` is actually set in `config.txt` — a huge number of "my sensor doesn't work" problems are actually just this line missing.

Overclocking

```
arm_freq=1800           # push the CPU clock speed higher than default (MHz)
over_voltage=2          # slightly increase voltage to keep the higher clock stable
```

Important warning to give students: Overclocking increases heat output significantly. It should always be paired with proper cooling (heatsink/fan) and thermal monitoring (see section 5). An unstable overclock

can cause random crashes or file corruption — never overclock a Pi running unattended without a way to check its temperature.

4. `raspi-config` — The Friendly Menu Version

Editing `config.txt` by hand works, but there's an easier, menu-based tool that does a lot of this safely for you:

```
sudo raspi-config
```

This opens a blue terminal menu:

```
Raspberry Pi Software Configuration
1 System Options      (hostname, password)
2 Display Options
3 Interface Options  (I2C, SPI, camera,
                     SSH, UART...)
4 Performance Options (overclock, GPU mem)
5 Localisation Options (timezone, locale)
6 Advanced Options   (expand filesystem)
```

Essentials every student should know how to do here:

- **Interface Options → I2C/SPI/Camera** — enable these instead of manually editing `config.txt` (safer, less chance of typos)
- **System Options → Hostname** — rename your Pi on the network
- **Performance Options → GPU Memory** — set the split without hand-editing the file
- **Advanced Options → Expand Filesystem** — makes sure your OS is using the *entire* SD card, not just the space the image originally needed (usually done automatically on first boot, but good to know it exists)

Why teach both methods: `raspi-config` is safer and recommended for beginners, but it's really just editing `config.txt` behind the scenes. Knowing the manual method matters for automation — for example, writing a setup script that configures 20 Pi boards identically without clicking through a menu on each one.

5. `vcgencmd` — Thermal Monitoring

`vcgencmd` is a Raspberry Pi-specific tool that talks directly to the Broadcom chip to report hardware status — most importantly, temperature and throttling state.

```
vcgencmd measure_temp
```

```
vcgencmd measure_temp
```

```
temp=52.3'C
```

Simple, direct CPU temperature reading.

Temperature	Status
Below 60°C	Comfortable
60–80°C	Getting warm, still generally fine
Above 80°C	Pi will start reducing performance to cool down
85°C	Hard throttling kicks in

`vcgencmd get_throttled` — The Command Students Always Forget About

```
vcgencmd get_throttled
```

```
throttled=0x0
```

This is one of the most useful — and most overlooked — diagnostic commands on the whole Pi. It reports a **hex code** where each bit means something specific:

Bit	Hex Value	Meaning
0	0x1	Under-voltage detected (right now)
1	0x2	ARM frequency capped (right now)
2	0x4	Currently throttled
3	0x8	Soft temperature limit active (right now)
16	0x10000	Under-voltage has occurred since boot
17	0x20000	Frequency capping has occurred since boot
18	0x40000	Throttling has occurred since boot
19	0x80000	Soft temp limit has occurred since boot

0x0 = perfectly healthy, nothing has ever gone wrong.

Real-world example: Your Pi seems to randomly freeze during an AI inference workload, but temperature readings look fine when you check. Running `vcgencmd get_throttled` and seeing `0x500005` tells you there's been an **under-voltage event** — meaning your power supply or USB cable isn't delivering enough current, not that your code or the AI model is broken. This single command has saved countless hours of debugging the wrong problem.

Quick monitoring loop (useful during a long AI training/inference run):

```
watch -n 2 vcgencmd measure_temp
```

This refreshes the temperature reading every 2 seconds automatically, so you can watch it live while your model runs.

Quick Recap

1. `apt` manages system software (like an app store); always `update` before `upgrade`. Use `apt-cache show` to inspect before installing, and `apt-mark hold` to freeze a package version.
 2. `pip3` manages Python packages. Use `--user` for personal installs, and on Bookworm, use a **virtual environment** instead of forcing installs with `--break-system-packages`.
 3. `/boot/config.txt` is the Pi's hardware control panel — GPU memory split, enabling I2C/SPI/UART/camera, and overclocking all live here.
 4. `raspi-config` is the safer, menu-driven way to change most of those same settings.
 5. `vcgencmd measure_temp` and `vcgencmd get_throttled` are your two go-to commands for diagnosing whether strange behavior is a software bug or a power/heat problem.
-

Self-Check Questions

1. What's the actual difference between `apt update` and `apt upgrade`? Why does order matter?
2. Why would you use `apt-mark hold` on a package, and give a realistic example.
3. On Raspberry Pi OS Bookworm, why does `pip3 install numpy` sometimes fail with an "externally-managed-environment" error — and what's the *recommended* fix (not the risky one)?
4. If you're building a headless AI project with no display connected, what `gpu_mem` value would you set in `config.txt`, and why?
5. A sensor connected over I2C isn't being detected at all. What's the very first thing you should check in `config.txt`?
6. Your Pi randomly freezes during a long inference job, but `measure_temp` shows a normal temperature. What command should you run next, and what might it reveal?